

Introduction to LLM Code Assistance Experiment



Introduction to the experiment process and rules



Brief explanation for problems and prompt patterns

Experiment Process

Introduction
10~15 min



Experiment
70 min



Survey
10~15 min





Experiment Rules

- You should record your screen using the given tool and share the recorded video in the survey as a link or file.
 - Please download and copy the given problems and submit the outcome code file & LLM communication logs as Zip file.
 - Please start the problem-solving from Task-1.
 - You can go to Task-2 after you achieve 121 PASS out of 128 test cases in Task-1, indicating 95% of PASS.
-



Experiment Rules (2/2)

- Please use the most familiar LLMs based on your experience.
 - Non-free versions of ChatGPTs, Copilot, or Claude would be highly recommended if you are confident to use one of them.
 - If LLMs keep returning unsatisfied outcomes three times, you can directly modify the codes. Otherwise, please use LLMs for coding.
 - If you want to ask question, please use Zoom chat or private message to organizers not to disturb other participants.
-



Task 1: E-Commerce Application

Key components overview

- User Class: Manages user registration and updates. -> Example provided.
 - Product Class: Holds product details such as name, price, and description.
 - ShoppingCart Class: Allows adding products and viewing cart contents.
 - Order Class: Processes user orders with items, address, and payment.
 - EcommerceApp Class: Manages the overall application flow.
-



Task 2: Smart Home Gym

Key components overview

- User Class: Manages user profiles, credentials, and exercise data. -> Example provided
 - Feed Class: Displays fitness articles.
 - MembershipRegistration Class: Handles new user registration with validation.
 - LoginSystem Class: Manages user authentication.
 - Calendar Class: Schedules workouts and tracks goals.
 - ExerciseSystem Class: Logs exercises and gives performance feedback.
 - Ranking Class: Calculates and shares user rankings.
-



Prompt Patterns in this Experiment

Reflection

**Alternative
approach**

**Cognitive
verifier**

No-pattern

Reflection Prompt Pattern

- Let LLM give you reasons behind why it gave you such answer to your prompt
- Also give you considerations that you might want to take in improving the answer

###Instruction###

When I ask a question, explain the reasoning behind your answer by breaking down the steps you took to arrive at it. Additionally, provide three alternative perspectives or considerations that could affect the accuracy or relevance of your answer. Once I evaluate those perspectives and give feedback, update your response accordingly to provide a more refined and complete answer.

###Question###

Write a function to iterate over each element in the list and add it to the sum if it is even.

###Reference###

Cognitive Verifier Prompt Pattern

- LLM asks multiple questions to make your prompt more specific answers.

###Instruction###

When I ask a question, generate three additional questions that would help you provide a more accurate answer. Once I answer those questions, combine the answers to give me a final response to my original question.

###Question###

Write a function to iterate over each element in the list and add it to the sum if it is even.

###reference###

Alternative Approach Prompt Pattern

- Find multiple different approaches to solve the problem, and give us the best one

###Instruction###

When I ask a question, I need you to give me three answers. Additionally, provide pros and cons for each answer. I also want you to choose the best answer from the three answers you gave. Once I evaluate your answer and give feedback, update your response accordingly to provide a more refined and complete answer.

###Question###

Write a function to iterate over each element in the list and add it to the sum if it is even.

###Reference###



Few-Shot Pattern

- Few-Shot prompt pattern is one of the most widely-used prompt pattern based on the reference context.
 - Please add the relevant context into the reference context, such as requirement specification, skeleton codes, test cases, etc.
-